LGWallRecon

Light-guide inner-wall reconstruction
from an entrance-face scan

User manual

Hengne Li (李衡讷)

South China Normal University

Hengne.Li@cern.ch Hengne.Li@m.scnu.edu.cn

12 November 2026

In one paragraph. LGWallRecon simulates a hollow conical light guide with a mirror wall by ray tracing, and searches for a wall shape for which the simulated response map matches a measured scan of the entrance face. For the scan shipped with the package the simulation and the measurement differ by 0.9 % in the inner aperture, against about 0.8 % point noise. The wall that achieves this is *not unique*: the scan fixes only a small part of the wall shape. Read the wall figure as one smooth, regularised solution that explains the scan, not as a survey of the real surface.

Contents

1 What the toolkit does **3**

2 Physics and model **6**

2.1 Geometry and coordinates . . . 6
2.2 Source, wall and detector . . . 6
2.3 The ideal cone . . . 6
2.4 The wall as a B-spline surface . . . 7
2.5 Ray–wall intersection . . . 7
2.6 Quasi-random rays and common random numbers . . . 8
2.7 How a patch of wall shows up in the scan . . . 8
2.8 The inversion . . . 9

3 Installation **10**

3.1 Requirements . . . 10
3.2 Python environment . . . 10
3.3 GPU (optional but recommended) . . . 10
3.4 Geant4 (optional) . . . 11

4 Quick start **11**

5 The pipeline, step by step **12**

5.1 `edge.py`: the entrance aperture . . . 12
5.2 `edge_spot.py`: the light spot . . . 13
5.3 `stageA.py`: the axisymmetric profile . . . 13
5.4 `refine.py`: re-gridding along z . . . 13
5.5 `kernel_modes.py`: the linear response . . . 13
5.6 `stageB.py`: the two-dimensional inversion . . . 14
5.7 `final.py`: the high-statistics comparison . . . 14
5.8 `pipeline.sh`: the whole chain . . . 15
5.9 Exports, figures and the report . . . 15

6 Validation tools **16**

6.1 Self-tests of the tracer . . . 17
6.2 Geant4 . . . 18
6.3 GPU against CPU . . . 18
6.4 Reciprocity . . . 19
6.5 Jacobian test . . . 19
6.6 Closure test . . . 20
6.7 Resolution study . . . 21
6.8 Two fits of the measured scan . . . 22
6.9 Further studies . . . 23

7 Outputs and file formats **23**
7.1 Input scan 23
7.2 Intermediate files in `out/` 24
7.3 Results 24

8 Adapting the toolkit to another light guide or scan **25**

9 Limits and interpretation **26**

10 Troubleshooting **27**

11 Resource policy: `tools/run_limited.sh` **28**

12 Author, contact, license and citation **28**

1 What the toolkit does

The light guide is a hollow cone, 17.5 mm in diameter at the entrance and 8 mm at the exit, 100 mm long, lined with reflective film. In the measurement a small diffuse light source is moved over the entrance face on a 100×100 grid with 0.2 mm step, and a PMT at the exit records the signal at each position. The result is a two-dimensional response map. It is not flat: it shows an off-centre dark spot, a bright ring and several bright patches, a few percent in size.

LGWallRecon answers two questions.

1. **Can a ray-tracing simulation reproduce this map?** Yes. An ideal cone responds almost identically at every source position (within $\pm 0.2\,\%$, Section 2.3), so the several-percent structure cannot come from an ideal cone; in this model it is attributed to the wall departing from the ideal shape. With a suitably deformed wall the simulated map agrees with the measured one to 0.9 % (rms, inner aperture $r < 8$ mm, 5029 scan points); the ideal cone misses by 5.4 %. The point noise of the measurement is about 0.8 %.
2. **Which wall does it?** The toolkit returns one: a deviation field $\delta(\varphi, z)$ from the CAD cone with a local unevenness of 13 µm rms, largest bump -61 µm, largest dent $+79$ µm, on top of an azimuthal mean deviation between -0.07 and -0.89 mm (the bore is narrower than CAD).

The honest headline. That wall is not the only one. In a closure test a known wall with seven bumps and dents was turned into a synthetic scan with the measured noise and sent through the same pipeline. The synthetic map was reproduced down to the noise (residual 0.78 %, $\chi^2/N = 0.95$), yet the recovered wall is only weakly correlated with the true wall (0.18): true local unevenness 48 µm, recovered 7 µm. A linear analysis explains why. The scan determines some combinations of the wall shape well and most others hardly at all, with no sharp boundary in between. Inside the 30 best-determined patterns the recovered and the true wall correlate at 0.97, inside the best 100 at 0.91, inside the best 150 at 0.70; but the test wall has only 1.4 %, 1.7 % and 4.0 % of its deformation power there. The rest is imprinted on the scan too weakly to be recovered.

The measured scan shows the same thing directly. The wall published with an earlier version of this toolkit, before two corrections of the tracer, still reproduces the scan when it is traced with the present tracer: to 0.92 %, against 0.90 % for the present wall. Yet the two walls are different. Their local unevenness correlates at 0.15 over the whole wall (0.80 in the first 20 mm behind the entrance, nothing further in), and their azimuthal mean profiles are 225 µm RMS apart (Section 6.8).

How to read the wall figure. It shows *one* regularised solution. The fit penalises unevenness and starts from an axisymmetric wall, so it tends towards a smooth wall, and it shows that little unevenness is enough to explain the scan. It is not the only wall that fits, it is not shown to be the least uneven one that fits, and it does not bound the unevenness of the real wall from below: reflectivity or detector non-uniformity could produce structure as well. It is not a survey of the real inner surface. Section 9 lists what extra measurements would be needed for that.

What the package contains:

- a specular ray tracer for a cone with a free-form wall, on the CPU (`numba`) and on the GPU (OpenCL), tracing the same rays;
- the inversion pipeline: aperture measurement, axisymmetric profile, linear-response kernels, damped least-squares inversion with exact residuals;
- validation tools: self-tests of the ray sequence, of the intersection and of the ideal cone, Geant4 cross-check, reciprocity test, GPU-against-CPU test, Jacobian test, closure test, resolution study;

- figure, table and report generators in Chinese and English, including a single-file web page with a live 3-D demonstration of rays in the guide.

Figure 1 shows the wall returned for the shipped scan and Figure 2 the comparison of the simulated with the measured map.

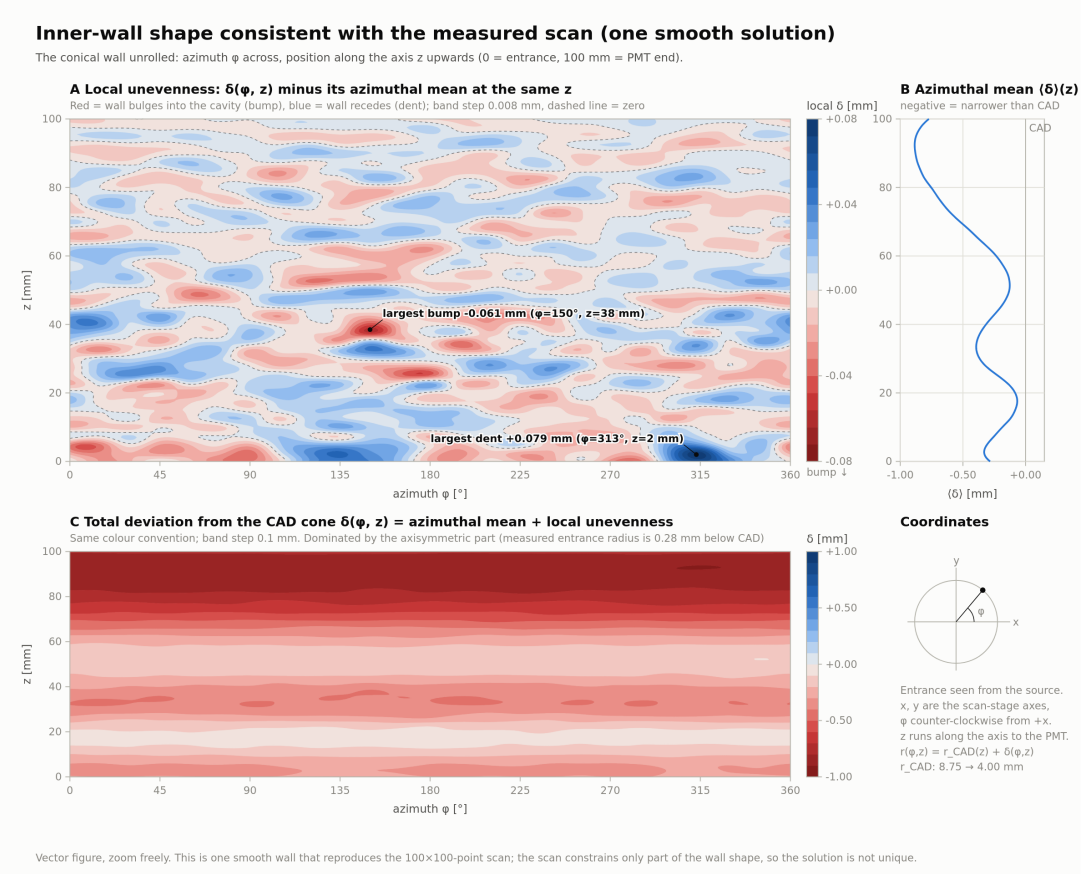


Figure 1: The wall figure (`inner_surface.svg`). A: local unevenness, the deviation minus its azimuthal mean. B: the azimuthal mean. C: the total deviation from the CAD cone. Red is a bump into the cavity, blue a dent.

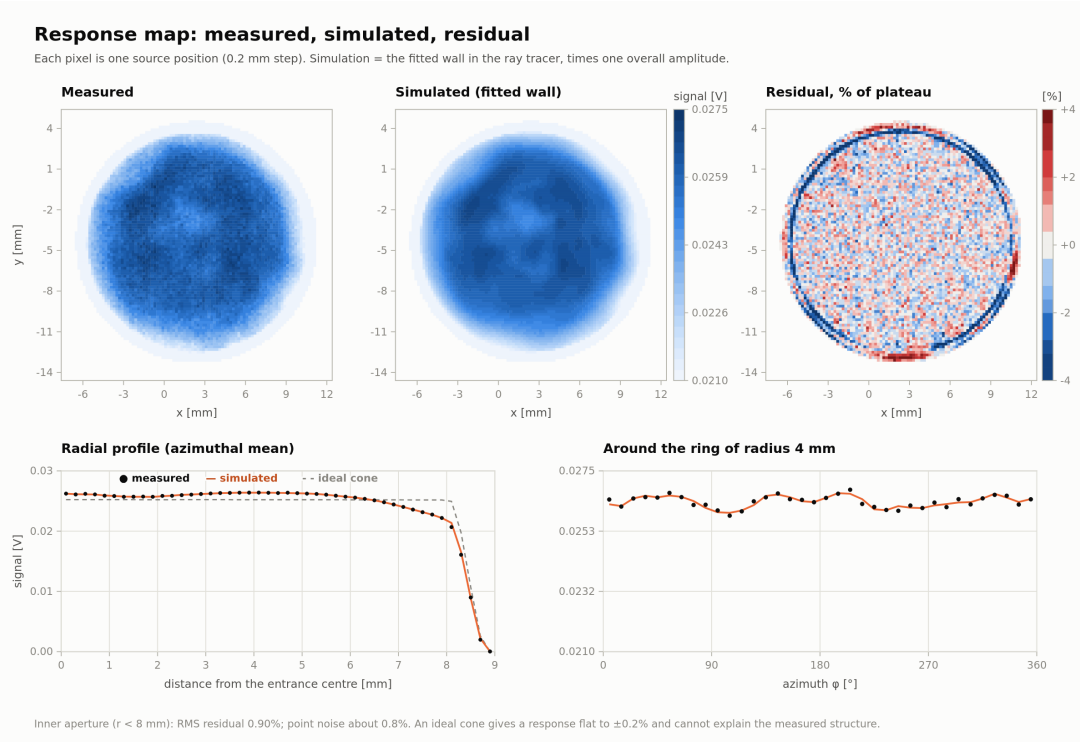


Figure 2: Measured map, simulated map and residual (`response_maps.svg`), with the azimuthally averaged radial profile and the signal around the ring of radius 4 mm.

2 Physics and model

2.1 Geometry and coordinates

All lengths are in millimetres. The z axis is the optical axis, $z = 0$ at the entrance plane and $z = L = 100$ at the exit plane where the PMT sits. The nominal (CAD) wall radius is

$$r_{\mathrm{CAD}}(z) = R_1 + (R_2 - R_1)\,\frac{z}{L}, \qquad R_1 = 8.75, \quad R_2 = 4.00,$$

a cone of half angle 2.72°. The real wall is written as

$$r(\varphi, z) = r_{\mathrm{CAD}}(z) + \delta(\varphi, z).$$

$\delta < 0$ means the wall bulges into the cavity (a bump); $\delta > 0$ means it recedes (a dent).

The transverse axes x, y are the scan-stage axes, with the origin moved to the measured centre of the entrance aperture. The azimuth φ is counted counter-clockwise from $+x$, looking at the entrance from the light-source side. Relating φ to the hardware requires knowing how the stage axes are oriented with respect to the light guide, and which of the four cells of the block was scanned; the toolkit cannot know either.

The CAD file `data/4cells_17.5mm_8mm_circle_to_circle_100mm-Cut.step` describes a block with four identical cells. Each cell is the cone above followed by a cylindrical pocket of 16.36 mm diameter and 10 mm depth that holds the PMT. Only the cone enters the simulation.

2.2 Source, wall and detector

Source A small spot in the entrance plane that emits like a Lambertian surface into the forward hemisphere. The spot is a uniform disc; its radius (0.34 mm for the shipped scan) is measured from the sharp edge of the scan. Rays that start outside the aperture are blocked.

Wall A perfect mirror: specular reflection with reflectivity 1. A reflectivity below 1 can be set (parameter `refl`) and was tried as a free parameter of the axisymmetric fit. Starting from the perfect-mirror fit, freeing the reflectivity moves it to 0.96, and χ^2 falls from 34 to 27 (40 points, one more parameter); the wall profile changes its shape but keeps its size. Reflectivity and profile are strongly degenerate and this scan cannot separate them, so results are quoted for a perfect mirror.

Detector Every ray that crosses the exit plane $z = L$ inside the wall is counted, whatever its angle and position. A ray that returns through the entrance plane is lost.

The simulated signal at source position (x, y) is $A\,T(x, y)$, where T is the fraction of all emitted light that reaches the exit and A is one overall amplitude (volts per unit transmission) fitted to the data. The absolute scale of the measurement is therefore not used.

2.3 The ideal cone

For a Lambertian source the ideal cone has a simple response. Averaged over the entrance area, the transmission equals the area ratio exactly,

$$\langle T_{\mathrm{ideal}} \rangle = \left(\frac{R_2}{R_1}\right)^2 = 0.2090.$$

This is conservation of étendue: a lossless concentrator cannot pass more than the area ratio of a Lambertian input, and the cone reaches that limit because every ray started backwards from the exit reaches the entrance.

At a single source position the transmission is only nearly that value. A reflection off the cone conserves the closest approach of a ray to the apex, so whether a ray gets through is decided by a sphere centred on the apex that passes through the exit rim. The exit, however, is a flat disc and not that spherical cap. The transmission therefore falls slowly from $+0.22\,\%$ at the centre to about $-0.14\,\%$ at the rim, relative to the area ratio. An earlier version of this manual called the response exactly flat; it is flat to $\pm 0.2\,\%$.

The tracer reproduces the area average within the tolerance of its self-test (2×10^{-4}, relative) and Geant4 agrees with the tracer along a radius (Section 6). The consequence for the inversion: the several-percent structure of the measured map cannot come from an ideal cone. In this model it is attributed to the wall being non-ideal; effects outside the model could contribute as well (Section 9).

2.4 The wall as a B-spline surface

$\delta(\varphi, z)$ is a tensor-product uniform cubic B-spline,

$$\delta(\varphi, z) = \sum_{i=0}^{N_\varphi - 1} \sum_{j=0}^{N_z + 2} c_{ij}\, B_i^{(\varphi)}(\varphi)\, B_j^{(z)}(z).$$

In φ the spline is periodic with $N_\varphi = 24$ knots at $\varphi_i = 2\pi i / N_\varphi$ (15° apart). In z there are N_z cells on $[0, L]$ and $N_z + 3$ coefficients; coefficient j is centred at $z_j = (j - 1)\, L / N_z$, so the first and the last one sit one cell outside the guide. The final fit uses $N_z = 40$ (2.5 mm cells, 43 coefficients per azimuth), which gives $24 \times 43 = 1032$ wall parameters. The spline is C^2, so the surface normal is continuous, which specular reflection needs.

2.5 Ray–wall intersection

A ray $\mathbf{p} + t\,\mathbf{d}$ hits the wall where $f(t) = \sqrt{x^2 + y^2} - r_{\mathrm{CAD}}(z) - \delta(\varphi, z)$ changes sign from negative to positive. The tracer finds the first such t in three steps.

1. *Bounding cones and the wall zone.* δ lies between two numbers δ_{lo} and δ_{hi} (sampled on a fine grid, with a small margin). The wall therefore lies between the cones $r_{\mathrm{CAD}} + \delta_{\mathrm{lo}}$ and $r_{\mathrm{CAD}} + \delta_{\mathrm{hi}}$, whose intersections with the ray are roots of a quadratic. Only the part of the ray inside this thin *wall zone* can hit the wall. The part inside the inner cone is skipped analytically, which leaves at most two intervals of t to search.

2. *Marching with the derivative.* Each interval is walked in steps of at most 0.5 mm, and both f and its derivative f' along the ray are sampled. A change of sign of f brackets a hit. If f' changes from positive to negative between two samples, f has a maximum in between; the maximum is located by regula falsi on f' and tested. A grazing hit that enters and leaves the wall within one step is therefore not skipped.

3. *Illinois regula falsi* on the bracket, to 10^{-10} mm in double precision (2×10^{-6} mm in single precision).

A hit can now be missed only if f has two extrema inside one 0.5 mm step, which needs structure far below the size of a spline cell (2.5 mm along z, at least 1 mm along the arc).

Earlier versions marched the whole bracket in 3 mm steps and looked for a sign change of f only; such a march can step over a grazing hit. `test_grazing_hit.py` holds a case that it missed: a ray 11 µm inside the nominal cone and a ring-shaped bump of 11.3 µm, which the ray penetrates by 0.3 µm over 0.7 mm of path. On the wall published with that version the 3 mm march, the present march and a 0.02 mm march gave identical transmissions for 2×10^7 rays, so the published

numbers were not affected. The outward normal is the gradient of $F = r - r_{\mathrm{CAD}}(z) - \delta(\varphi, z)$,

$$\mathbf{n} \;\propto\; \hat{\mathbf{r}} \;-\; \frac{1}{r}\frac{\partial \delta}{\partial \varphi}\,\hat{\boldsymbol{\varphi}} \;-\; \Big(\frac{R_2 - R_1}{L} + \frac{\partial \delta}{\partial z}\Big)\hat{\mathbf{z}},$$

with the derivatives taken analytically from the spline, and the ray is reflected as $\mathbf{d}' = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\,\mathbf{n}$. A ray ends when it crosses $z = L$ (detected) or $z = 0$ (lost).

2.6 Quasi-random rays and common random numbers

For a Lambertian source the direction cosines (d_x, d_y) are uniform in the unit disc. Only directions with $\sin\theta \le s_{\max}$ are sampled, because steeper rays never reach the exit (for the fitted wall nothing is lost with $s_{\max} = 0.5$; the pipeline uses 0.55), and the result is scaled by $s_{\max}^2$.

Rays are quasi-random rather than random. The four numbers that define a ray (two for the direction, two for the position inside the spot) come from the R_4 additive recurrence $u_j(k) = \mathrm{frac}(s_j + k\,g^{-j})$, $j = 1 \ldots 4$, with $g = 1.1673039782614187$, the real root of $x^5 = x + 1$. The numbers $1, g^{-1}, \ldots, g^{-4}$ are linearly independent over the rationals, so the points fill the unit 4-cube evenly. The recurrence is evaluated in 32-bit integer arithmetic so that the CPU and the GPU produce the same rays. The two extra coordinates of the optional blur of the spot use $\sqrt{2} - 1$ and $\sqrt{3} - 1$. Each source position has its own shifts s_j (a Cranley–Patterson rotation derived from a hash of the point index and a seed).

Corrected in this version. Earlier versions used the root of $x^4 = x + 1$, for which $g^{-3} + g^{-4} = 1$ exactly. The two spot coordinates were then locked together and the spot was sampled along a spiral instead of over the disc, with its centroid displaced by 0.12 spot radii in a direction that changed from point to point. This acted as extra model noise (0.22 % of T instead of 0.10 % at 20 000 rays per point), not as a bias (mean shift 4×10^{-5}). `test_sampler.py` now checks that no integer relation with small coefficients holds between the constants, that every pair of coordinates fills a 32×32 grid evenly, and that the centroid of the sampled spot sits at its centre.

The chain was rerun from the start with both corrections (the march of Section 2.5 and this one). The fit quality changed little (χ^2/N from 1.31 to 1.27), but the wall that came out is a different one (Section 6.8).

Two properties follow from the quasi-random rays.

- The noise of T falls faster than $1/\sqrt{N}$: with 20 000 rays per point it is about 0.1 % of T, where independent random rays would give about 1.5 %.
- For a fixed seed the map is a deterministic function of the wall. Differences between two walls are then computed with *common random numbers*: most rays have the same fate in both, and the noise of the difference is far below the noise of either map. All finite differences and all fit iterations rely on this.

2.7 How a patch of wall shows up in the scan

A small deformation at height z and azimuth φ does not change the response near the point on the entrance face "below" it. It changes the fate of the marginal rays, those that just make it to the exit or just fail to, and for a given wall patch these rays come from a thin arc of source positions. The arc is typically 0.5 mm wide, and its position and radius change quickly with z: patches near the entrance affect sources at the adjacent rim, patches around mid-length draw arcs and rings a few millimetres from the centre, patches near the exit give diffuse rings. The picture resembles tomography, with each scan pixel summing wall deformations along a curve.

Two practical consequences: the response is very sensitive (a deformation of order 0.03 mm changes the map by a few percent), and it needs a fine axial resolution of the wall model (2.5 mm cells) to place the arcs correctly.

2.8 The inversion

The fit minimises the objective

$$Q(c, A) = \sum_n \frac{\big(d_n - A\,T_n(c)\big)^2}{\sigma^2} + \lambda^2\,\|Pc\|^2 + (0.05\,\lambda)^2\,\|c - c_{\mathrm{base}}\|^2 + \sum_i \frac{\big(\delta(\varphi_i, 0) - \delta_{\mathrm{ap}}(\varphi_i)\big)^2}{(0.005\ \mathrm{mm})^2}.$$

d_n are the pedestal-subtracted scan values inside the fit radius (8 mm by default), σ combines the point noise (2.0×10^{-4} V) with the Monte Carlo noise of the model, P takes first differences of the coefficients along φ and z (a smoothness penalty), c_{base} is the axisymmetric base wall, and the last term ties the wall radius at $z = 0$ to the aperture radius measured from the edge of the scan (in a closure test, to the aperture of the synthetic wall). The first term is the data χ^2; the printed χ^2/N refers to it alone. The second and third terms penalise unevenness and distance from the base wall, in the metric of the spline coefficients; Q is not the surface RMS of the wall under a fit tolerance.

$T_n(c)$ is not differentiable in closed form, and a brute-force finite-difference Jacobian is too noisy for single spline coefficients. The package uses two devices.

Fourier-mode kernels. Around an axisymmetric base wall the response to an azimuthal mode $\delta c_{ij} = a\cos(m\varphi_i)$ at one z knot is a sum of a few harmonics $\sum_q A_{jmq}(\rho)\cos(q\psi)$ of the source azimuth ψ, with $q \in \{m,\, N_\varphi - m,\, N_\varphi + m\}$. One run perturbs the whole circumference, so the signal is strong while the wall slopes stay small, and projecting on $\cos(q\psi)$ averages the noise over all source angles. Central differences remove the second-order term. The Jacobian for any single coefficient follows by Fourier synthesis and rotation. Against exact finite changes of the map (Section 6.5) these kernels reach a correlation of 0.84–0.99 and a slope of 0.74–1.03 for non-axisymmetric perturbations of 10–100 µm RMS. For changes of the azimuthal mean profile they are poor: the correlation is 0.44–0.50 for a gentle bump and 0.13–0.27 for a sharp step. A smooth deformation is the small remainder of large single-knot responses that cancel, so errors of a few percent in the single-knot kernels become errors of order one for smooth shapes.

Damped iterations with exact residuals. Each Levenberg–Marquardt step solves the linearised problem with a damping term $\mu\|\Delta c\|^2$, then evaluates the true Q with a full ray trace. The step is kept if Q falls (and μ is halved); otherwise μ is multiplied by four and the step is recomputed, up to eight times. A Broyden rank-one update corrects the Jacobian with what each trial step actually did to the map. Because the residual is always exact, every accepted step really lowers Q, whatever the errors of the kernels.

That does not mean the best fit is reached. The iteration stops when eight successively more damped trial steps all fail to lower Q, which is a practical stopping rule and not a proof of a minimum. The result is one regularised solution; it depends on the start wall, on the regularisation weights and on the approximate Jacobian. Two earlier fits of the shipped scan, with earlier versions of the code, ended at $\chi^2/N = 1.48$ and 1.31; the present one ends at 1.27.

The axisymmetric part of the wall is large (tenths of a millimetre) and non-linear, so the chain first fits an axisymmetric profile alone (Stage A), then runs two rounds of kernels and inversion, the second with kernels recomputed around the updated azimuthal mean.

3 Installation

3.1 Requirements

Component	Needed for
Python $\geq$ 3.10 with `numpy`, `scipy`, `matplotlib`, `numba`, `contourpy`, `pillow`	everything
`pyopencl` and an OpenCL GPU	fast tracing (optional, `requirements-gpu.txt`; without it use `LG_GPU=0`)
Geant4 11.x, CMake, a C++17 compiler	the independent cross-check in `g4/` (optional)
Node.js	the animated ray figure `ray_animation.svg` (optional)
TeX Live with `xelatex` and `ctex`	rebuilding this manual (optional)
Firefox	rendering the manual figures from SVG (optional)
Linux with `systemd` user sessions	the resource wrapper `tools/run_limited.sh`

3.2 Python environment

```
git clone <repository url> LGWallRecon
cd LGWallRecon
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements-gpu.txt   # with the GPU tracer (adds
    pyopencl)
.venv/bin/python -m pip install -r requirements.txt       # or: CPU only
```

`requirements.txt` lists what a CPU-only installation needs; `requirements-gpu.txt` includes it and adds `pyopencl`, which needs a working OpenCL driver.

The scripts expect the interpreter at `.venv/bin/python` relative to the package root. The shell drivers accept another interpreter through the variable `PY`.

3.3 GPU (optional but recommended)

The tracer runs on any OpenCL device that `pyopencl` can see. Check with

```
clinfo -l
.venv/bin/python -c "import pyopencl as cl; print([d.name for p in cl.get_platforms()
    for d in p.get_devices()])"
```

The development machine has an AMD Radeon 8060S (Ryzen AI Max+ 395) driven through ROCm 7.2; the device appears as `gfx1151` on the platform "AMD Accelerated Parallel Processing". On AMD hardware `nvidia-smi` is absent and says nothing about GPU availability; use `clinfo` or `rocminfo`.

Measured speeds for the fitted wall:

Configuration	Rays per second
GPU, single precision (default)	about 2.4×10^7 (measured while the machine was busy)
GPU, double precision	roughly 10–30 times slower than single
CPU, double precision (`numba`)	of order 10^5 per core

The GPU version traces tens of millions of rays per second, roughly an order of magnitude faster than the CPU version on all cores of this machine. The wall-zone march with the derivative (Section 2.5) costs about a factor 1.6 on the GPU against the plain 3 mm march of earlier versions, which reached 3.8×10^7 rays per second. The kernel is single precision; its result for every source point was checked against the CPU in double precision (Section 6). Without a GPU everything still runs with `LG_GPU=0`; expect the full chain to take many hours instead of about 50 minutes.

Integrated GPUs share the system memory, and a cgroup memory limit does not cap GPU allocations. The tracer keeps its buffers at a few megabytes and submits work in slices of about half a second, so it neither exhausts memory nor trips the driver watchdog.

3.4 Geant4 (optional)

```
source <geant4 prefix>/bin/geant4.sh
cd g4 && mkdir -p build && cd build
cmake .. -DCMAKE_PREFIX_PATH=<geant4 prefix>
make -j4
```

This builds `g4/build/lgsim`. No Geant4 data sets beyond the standard installation are needed; the program uses a minimal physics list with optical photons, transportation and the optical boundary process only.

4 Quick start

```
cd LGWallRecon
./run_all.sh
```

`run_all.sh` first runs three self-tests of the tracer (`test_sampler.py`, `test_grazing_hit.py`, `test_ideal_cone.py`), then measures the aperture, runs the reconstruction chain on `data/scan_summary.csv`, compares the GPU with the CPU on the fitted wall (if the GPU is on), compares the fitted wall with the wall of an earlier fit of the same scan (`compare_walls.py`, Section 6.8), exports the wall, draws the figures in both languages and assembles the report pages. Every heavy step goes through `tools/run_limited.sh` (Section 11). The drivers run with `set -euo pipefail`: a failing step or self-test stops the run.

```
./run_all.sh --with-checks
```

also runs the closure test with the resolution study, the reciprocity test, the Jacobian test, the axisymmetric fit with a free mirror reflectivity and, if Geant4 is installed, the Geant4 cross-check.

Measured run times with the GPU, on a machine that was busy with other jobs: 48 minutes for the chain from the aperture to the final map, most of it in the two kernel computations (about 22 minutes each). The checks take about two hours more: the closure test about 60 minutes, the reciprocity test about 22 minutes, Geant4 (both runs, with the mesh) about 40 minutes. An idle machine is faster.

Where the results appear:

Path	Content
`results/en/inner_surface.svg`, `results/zh/...`	the wall figure (vector, zoomable)
`results/<lang>/response_maps.svg`	measured map, simulated map, residual, profiles
`results/<lang>/closure.svg`	closure test (after `closure.sh`)
`results/<lang>/ray_animation.svg`	animated side view of traced rays
`results/wall_delta.csv`	$\delta(\varphi, z)$ on a $1^\circ \times 0.5\,\mathrm{mm}$ grid
`results/wall_coeffs.json`	the B-spline coefficients
`results/numbers.json`	every number quoted in the report and in this manual
`docs/LGWallRecon_report_en.html`, `..._zh.html`	single-file report pages with the 3-D demonstration
`out/`	intermediate `.npz` and `.log` files (not versioned)

To run the stages by hand, change into `src/` first; the scripts use paths relative to it.

```
cd src
R=../tools/run_limited.sh; PY=../.venv/bin/python
$R $PY edge.py && $R $PY edge_spot.py      # aperture and light spot -> ../out/edge.npz
$R ./pipeline.sh data                      # the reconstruction chain, log in ../out/
    pipeline_data.log
$R ./make_report.sh ../out/final_data.npz  # exports, figures, report pages
```

5 The pipeline, step by step

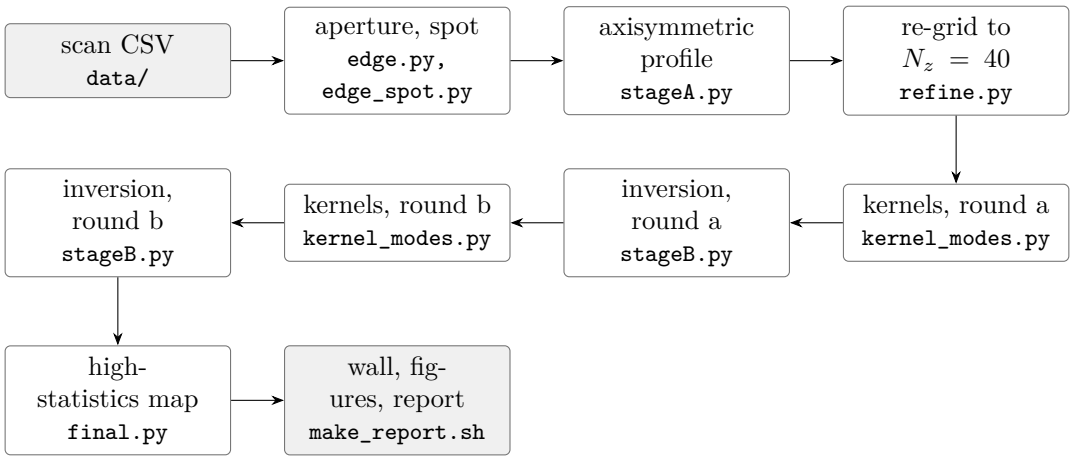


Figure 3: The reconstruction chain. `pipeline.sh` runs the boxes from `stageA.py` to `final.py`.

All commands below are run from inside `src/`, through `../tools/run_limited.sh`. Environment variables used by the scripts are collected in Table 1.

5.1 `edge.py`: the entrance aperture

```
python edge.py
```

The edge of the scan is sharp, so the aperture can be measured directly, without ray tracing. The script takes the pedestal as the median signal far outside the aperture ($r > 10\,\mathrm{mm}$), then fits the

radial edge profile in 48 azimuthal sectors with a plateau level, a slope and an error-function edge. The centre is moved until the $m = 1$ Fourier component of the edge radius vanishes. Output: `../out/edge.npz` with the centre, the pedestal and the Fourier coefficients ($m = 0 \ldots 8$) of the edge radius $r_{\mathrm{ap}}(\varphi)$, and a diagnostic plot `../out/edge.png`.

For the shipped scan the mean radius is 8.47 mm, that is 0.28 mm below the CAD value of 8.75 mm, with $m = 2$ and $m = 4$ components of about 0.03 mm.

The first guess of the centre and the radial window of the edge fit are constants near the top of the script; change them for a different aperture (Section 8).

5.2 `edge_spot.py`: the light spot

```
python edge_spot.py
```

A global fit of all edge points compares a Gaussian spot with a uniform disc. The disc describes the edge better (the signal has no tail outside the aperture). The script adds the disc radius (0.34 mm) and a small correction of the mean aperture radius to `../out/edge.npz`. All later stages read the spot from that file.

5.3 `stageA.py`: the axisymmetric profile

```
LG_TAG=data python stageA.py 10 fix 3.0
LG_TAG=refl LG_START=../out/A_data.npz python stageA.py 10 refl 3.0
```

Arguments: number of z cells N_z (10), `fix` or `refl` (fit the mirror reflectivity as well), weight of the curvature penalty (3.0). With `LG_START=<A_*.npz>` the fit starts from the profile of an earlier Stage-A fit and from a reflectivity of 1, so a fit with the reflectivity free can only improve on the perfect-mirror fit it starts from and the two χ^2 can be compared (second line; this is what `run_all.sh --with-checks` runs). The reflectivity is capped at 1 and its derivative is taken as a backward difference. The script fits an axisymmetric wall offset $\delta_0(z)$ and the amplitude to the azimuthally averaged radial profile of the scan ($\rho = 0.1 \ldots 7.9$ mm), by Levenberg–Marquardt with finite differences and common random numbers (120 000 rays per radius). The radius at $z = 0$ is tied to the measured aperture. Output: `../out/A_<tag>.npz` and a plot.

How to judge it: the printed "data" χ^2 should fall by more than an order of magnitude from the start value, and the residual plot should stay within about $\pm 1\,\%$ of the plateau. The profile itself is *not* well determined: different runs return visibly different $\delta_0(z)$ with the same fit quality (Section 9).

5.4 `refine.py`: re-gridding along z

```
python refine.py ../out/A_data.npz 40 ../out/start_data.npz
```

Re-expresses a wall on a different number of z cells. The input is either a Stage-A result (the profile is copied to all 24 azimuthal knots) or a fitted wall. For nested grids ($10 \to 40$) the conversion is exact.

5.5 `kernel_modes.py`: the linear response

```
python kernel_modes.py ../out/start_data.npz 24 dataa 12000 0.06
```

Arguments: a wall file (its azimuthal mean becomes the base wall), N_φ, a tag, rays per source point (12 000), mode amplitude a in mm (0.06; mode m uses $\min(a, 0.18/m)$, which caps the wall slope of the high modes; in `pipeline.sh` the amplitude is set with `LG_KAMP`). Sources sit on a polar grid of 57 radii (0.075 to 8.475 mm) and 49 angles on the half circle. For each of the 43 z knots and each mode $m = 0 \ldots 12$ the script traces the wall with $+a$ and $-a$ and stores the harmonic amplitudes $A_{jmq}(\rho)$. Output: `../out/kmodes_<tag>.npz`. Run time: about 22 minutes on the GPU of a busy machine; one progress line per z knot.

The module also provides `jacobian(kn, rho, psi)`, which assembles the Jacobian for arbitrary source positions from the stored amplitudes.

5.6 `stageB.py`: the two-dimensional inversion

```
LG_MU=3000 python -u stageB.py dataa dataa 1.0 40 ../out/start_data.npz
```

Arguments: kernel tag, output tag, regularisation weight λ (1.0), maximum number of iterations (40), start wall. Output: `../out/fitB_<outtag>.npz`, rewritten after every accepted iteration, so an interrupted run can be continued by passing that file as the start wall.

One line is printed per iteration:

```
it 7: chi2/N 1.518  rms resid 1.016 %  A 0.14979  |delta-base| rms 0.063 max 0.199
    rough 11.291
```

- `chi2/N`: data χ^2 per scan point, with σ = point noise $\oplus$ model noise. A value near 1 means the map is fitted to the noise. The shipped scan ends at 1.27: a few tenths of a percent of structure on a 1 mm scale is not explained by this fit. The synthetic scan of the closure test reaches 0.95, so the chain can reach the noise when the truth lies inside the model; whether a better fit of the shipped scan exists inside the model is not known. χ^2/N is not a calibrated goodness of fit: the effective number of fitted parameters is unknown.
- `rms resid`: root mean square of data minus model, in percent of the mean signal.
- `|delta-base|`: rms and maximum of the coefficients relative to the base wall, in mm.
- `rough`: the value of $\|Pc\|^2$.

The run ends after the given number of iterations or with "`no improving step; stop`". That message means that eight successively more damped trial steps, down to sub-micrometre size, all failed to lower the objective. This is the stopping rule and the normal way for the second round to finish; it does not prove that a minimum has been reached. If it appears in the very first iteration with a large χ^2, the damping start value is too small for the state of the wall; raise `LG_MU` (the chain uses 3000 for round a and 20 000 for round b).

With `LG_VERB=1` every trial step is printed with its size, the χ^2 predicted by the linear model, the χ^2 actually achieved and the objective. A large gap between the two is the signature of kernels that no longer describe the wall; recompute them around the current azimuthal mean.

5.7 `final.py`: the high-statistics comparison

```
python final.py ../out/fitB_datab.npz data 40000
```

Traces the whole scan grid with the fitted wall (two seeds, to measure the model noise) and with the ideal cone of the measured aperture radius, fits the amplitude, prints the summary numbers and writes `../out/final_<tag>.npz` and a diagnostic PNG. All figures and the report are built from this file.

5.8 `pipeline.sh`: the whole chain

```
../tools/run_limited.sh ./pipeline.sh data
```

Runs Stage A, the re-gridding, two rounds of kernels and inversion, and `final.py`, with the settings given above. The tag names all intermediate files (`A_data`, `start_data`, `kmodes_dataa`, `fitB_dataa`, `kmodes_datab`, `fitB_datab`, `final_data`). `edge.py` and `edge_spot.py` must have been run before. With `LG_SYNTH=<npz>` the chain runs on a synthetic scan instead of the measured one.

The script first removes the files of an earlier run with the same tag, writes the full output of every step to `../out/pipeline_<tag>.log` and stops at the first step that fails, so a failed step cannot be papered over by old files. Only a few summary lines are printed to the terminal.

5.9 Exports, figures and the report

Command	Result
`export_wall.py <final npz> <dir>`	`wall_delta.csv`, `wall_coeffs.json`
`compare_walls.py <final npz> <earlier wall_coeffs json>`	`../out/compare_prev.json`: this wall against the wall of an earlier fit (Section 6.8); run by `run_all.sh`
`collect_numbers.py <final npz> <results dir>`	`numbers.json` (also reads the results of the checks that are present and marks whether each belongs to this wall)
`make_figures.py <final npz> <dir> <lang>`	`inner_surface.svg`, `response_maps.svg`
`make_closure_fig.py <synth npz> <final npz> <dir> <lang>`	`closure.svg`, `closure_numbers.json`
`make_ray_svg.py <final npz> <dir> <lang>`	`ray_animation.svg` (needs Node.js)
`build_page.py <final npz> <results dir> <out html> <lang> [github] [gitlab]`	one single-file report page
`make_report.sh [final npz] [github url] [gitlab url]`	all of the above for `zh` and `en`; the URLs default to the home of the toolkit

`<lang>` is `zh` or `en`. All strings shown in figures and pages live in `web/i18n/zh.json` and `web/i18n/en.json`; the code contains no display text.

Table 1: Environment variables.

Variable	Used by	Meaning (default)
`LG_GPU`	`lgtrace.py`	1: trace on the GPU; 0: CPU. The wrapper sets 1; without the wrapper the default is 0
`LG_CPUS`	`run_limited.sh`	number of CPU cores (chosen from the idle cores, 2–24)
`LG_MEM`	`run_limited.sh`	hard memory limit of the job (`6G`)
`LG_SCAN`	`load_scan.py`	path of the scan CSV (`data/scan_summary.csv`)
`LG_SYNTH`	`stageA.py`, `stageB.py`, `final.py`, `load_scan.py`	`.npz` with a synthetic scan that replaces the measured one, and with the aperture of its wall
`LG_TAG`	`stageA.py`	name of the Stage-A output, `A_<tag>.npz`
`LG_START`	`stageA.py`	earlier `A_*.npz` whose profile is the start (none: flat start)
`LG_RCUT`	`stageB.py`	fit radius in mm (8.0)
`LG_KAMP`	`pipeline.sh`	finite-difference amplitude of the kernels in mm (0.06)
`LG_NRAY`	`stageB.py`	rays per scan point in each iteration (20 000)
`LG_MU`	`stageB.py`	start value of the damping (3000)
`LG_BROYDEN`	`stageB.py`	1: secant correction of the Jacobian (1)
`LG_VERB`	`stageB.py`	1: print every trial step (0)
`PY`	shell drivers	Python interpreter (`../.venv/bin/python`)
`LG_GITHUB_URL`, `LG_GITLAB_URL`	`make_report.sh`	repository URLs printed in the report pages (the home of the toolkit)

6 Validation tools

The checks answer three different questions: is the tracer right, can the inversion recover a known answer, and how much does the result depend on the assumptions. Table 2 collects the results for the shipped scan.

Table 2: Validation results. An entry "n/a" means that the check was not run for this reconstruction.

Check	Result	Notes
Ray sequence	fills the unit cube	`test_sampler.py`; stops `run_all.sh` if it fails
Grazing hit	found; fine march changes no ray	`test_grazing_hit.py`; stops `run_all.sh` if it fails
Ideal cone, tracer	centre +0.23 %, rim −0.13 %	relative to $(R_2/R_1)^2$; the area average equals it within 2×10^{-4}
Geant4, ideal cone	$T = 0.2094 \pm 0.0002$	ratio to the tracer 1.0012 ± 0.0011; 30 points × 40000 photons
Geant4, fitted wall	ratio 1.0007 ± 0.0015	120 points × 4000 photons, normalised scatter 1.00
GPU against CPU	same result for every point	double precision: equal to rounding error; single precision: mean relative difference $-2.0 \times 10^{-6} \pm 2.8 \times 10^{-6}$, scatter 6.2×10^{-5}
Reciprocity	ratio 1.003 ± 0.009	compatible with 1
Jacobian test	correlation 0.84–0.99, slope 0.74–1.03	non-axisymmetric perturbations of 10–100 µm RMS. Mean profile: gentle bump 0.44–0.50, sharp step 0.13–0.27: the kernels are poor there
Closure test, the map	residual 0.78 %	fitted to the noise, $\chi^2/N = 0.95$
Closure test, the wall	correlation 0.18	48 µm true, 7 µm recovered, difference 47 µm (all RMS)
Two fits of the measured scan	correlation 0.15	earlier wall fits to 0.92 %, present wall to 0.90 %; profiles 225 µm RMS apart
Free reflectivity	$R = 0.96$	started from the perfect-mirror fit; χ^2 falls from 34 to 27 (40 points); degenerate with the profile

Which wall a check belongs to. Every result file of a check carries a fingerprint of the wall it was run on (a hash of the rounded spline coefficients; for the closure test and the resolution study, of the axisymmetric profile their truth is built from). `collect_numbers.py` compares it with the wall of the final result and stores `current: true` or `false` with each check in `numbers.json`. The report page labels a check that is left over from an earlier reconstruction as such, so an old check cannot pass for a check of a new wall.

6.1 Self-tests of the tracer

```
LG_GPU=0 python test_sampler.py        # the quasi-random rays fill their unit cube
LG_GPU=0 python test_grazing_hit.py    # no grazing wall hit is skipped
LG_GPU=0 python test_ideal_cone.py     # ideal cone: area-averaged T = (R2/R1)^2
```

Each script prints PASS or FAIL per criterion and exits non-zero on a failure; `run_all.sh` runs all three first.

- `test_sampler.py`: no integer relation $\sum_j m_j \alpha_j$ with $|m_j| \leq 4$ between the six constants of the ray sequence comes closer to an integer than 10^{-6}; every pair of coordinates fills a 32×32 grid evenly with 65 536 points; the centroid of the sampled disc spot is at its centre.
- `test_grazing_hit.py`: the grazing hit of Section 2.5 is found, by the CPU tracer and (if Node.js is installed) by `web/lgtrace.js`; on a rough random wall not a single ray out of

3×10^6 changes between the production march of 0.5 mm and a march of 0.05 mm.

- `test_ideal_cone.py`: the analytic-cone path and the spline path with zero deformation give the same numbers; the area-weighted mean transmission equals $(R_2/R_1)^2$ within 2×10^{-4} (relative); the centre lies +0.22 % and the rim about −0.14 % from it (measured: +0.23 % and −0.13 %); cutting the launch cone at $\sin\theta = 0.7$ loses nothing. The script writes `../out/ideal_cone.json`.

6.2 Geant4

`g4/lgsim` traces optical photons in a hollow cavity whose wall is an optical surface of type `dielectric_metal`, polished, with the given reflectivity. The cavity is either the ideal `G4Cons` or a `G4TessellatedSolid` built from a wall mesh.

```
lgsim <mesh.txt | ideal> <points.txt> <photons per point> <out.txt> [spot] [smax] [refl
    ] [seed]
```

`spot` > 0 is a Gaussian sigma, < 0 a uniform disc of radius $-$`spot` (mm); `smax` restricts the launch cone as in the tracer. The output has one line per source point: x, y, detected, emitted, T.

```
cd g4
./run_final.sh ../out/fitB_datab.npz [NPROC] [photons per point]
```

`run_final.sh` does both comparisons, the fitted wall and the ideal cone, with the light spot and the mirror reflectivity of the fit: `export_mesh.py` writes them next to the mesh (`<mesh>.params`: spot, `smax`, reflectivity), and both `lgsim` and `g4_compare.py` read that file.

Fitted wall. The mesh has 1440×251 vertices, about 720 000 triangles; facets are 0.25° wide and 0.4 mm long, fine enough that faceting does not show at the present precision. Tracking in the tessellated solid costs about 10 ms per photon (the ideal `G4Cons` needs 16 µs), which is why Geant4 is used for checking and not for fitting. The 120 source points are split into `NPROC` lists (by default the number of idle cores, at most 12), one single-threaded `lgsim` process each, of about 0.7 GB. All processes run inside *one* resource scope with `NPROC` cores and `NPROC` + 2 GB in total (Section 11). The script checks the exit status of every process and that all source points came back; otherwise it stops. With 12 processes, 4000 photons per point take roughly ten minutes.

Ideal cone. `G4Cons`, 30 source points along a radius, 40 000 photons each.

```
python g4_compare.py <fit npz | ideal> <params.txt> <g4 out files...>
```

`g4_compare.py` traces the same points with the tracer at high statistics (400 000 rays per point, on the CPU) and writes the ratio of the mean transmissions with the binomial error of the Geant4 counts, the normalised scatter (1 for pure statistics), the correlation and the fingerprint of the wall to `../out/g4_compare.json` or `../out/g4_ideal.json`.

Result for the fitted wall: ratio 1.0007 ± 0.0015, normalised scatter 1.00, correlation 0.96. For the ideal cone: $T = 0.2094 \pm 0.0002$, ratio to the tracer 1.0012 ± 0.0011.

6.3 GPU against CPU

```
python test_gpu_vs_cpu.py
```

Traces one fitted scan point in ten (503 points, 8000 rays each) with the same wall, the same spot and the same rays on the CPU (double precision), on the GPU in double precision and on the GPU in single precision. The quantity compared is the transmission of each source point, a sum over its rays, not the individual rays.

- Double precision: the GPU must reproduce the CPU value of every point within two rays. It does so to rounding error.
- Single precision, which the fit uses: the mean difference must vanish within four times its statistical error and the point-to-point scatter must stay below 10^{-3} of T. Measured: mean relative difference $-2.0 \times 10^{-6} \pm 2.8 \times 10^{-6}$, scatter 6.2×10^{-5}; a few rays per point change fate, as expected for marginal rays.

The script prints the throughput of each, writes `../out/gpu_check.json` and exits non-zero if a criterion fails. `run_all.sh` runs it after the reconstruction when the GPU is on.

6.4 Reciprocity

```
LG_GPU=0 python test_reciprocity.py ../out/fitB_datab.npz
```

For a lossless specular guide lit with the same Lambertian radiance from both ends, the flux from entrance to exit equals the flux from exit to entrance. A wrong intersection, or a normal that is inconsistent with the surface, breaks this identity. The script traces both directions on the CPU for a smooth wall, two random rough walls and the fitted wall. Every case must give a ratio compatible with 1 within four standard deviations, otherwise the script exits non-zero. For the fitted wall the ratio is 1.003 ± 0.009 (written to `../out/recip.json`). For the ideal cone both fluxes also equal the exit area $\pi R_2^2 = 50.3\,\mathrm{mm}^2$, as étendue requires; with a deformed wall some rays started backwards from the exit turn round, and the common flux is smaller than the exit area.

6.5 Jacobian test

```
python test_jacobian.py datab ../out/fitB_datab.npz
```

Compares the kernel prediction $J\,\Delta c$ with the exact change of the map (200 000 rays per point, common random numbers), around the base wall and around the fitted wall, for seven perturbations: five random smooth fields of 0.01–0.1 mm rms without an azimuthal mean (random coefficients smoothed by two or by six passes of a three-point average: short and long wavelength), and two axisymmetric ones, a sharp step and a gentle bump. The script prints the correlation of the predicted with the exact change (means removed) and the slope (exact over predicted, means not removed). `run_all.sh --with-checks` runs it and keeps the printout in `../out/test_jacobian.log`; the script also writes `../out/jacobian.json`, which `collect_numbers.py` copies into the block `jacobian` of `numbers.json`. The table is the run on the wall of this result, with the kernels the fit used (finite-difference amplitude 60 µm); its log is tracked as `results/checks/test_jacobian_amp060.log`.

	Base wall		Fitted wall	
Perturbation	corr.	slope	corr.	slope
10 µm RMS, short wavelength	0.97	1.03	0.96	1.02
30 µm RMS, short wavelength	0.96	0.93	0.89	0.90
60 µm RMS, short wavelength	0.90	0.74	0.84	0.74
30 µm RMS, long wavelength	0.99	0.98	0.94	0.85
100 µm RMS, long wavelength	0.95	0.91	0.95	0.91
axisymmetric step, +50 µm for $z > 50$ mm	0.27	0.95	0.13	0.93
axisymmetric bump, 30 µm, $\sigma = 10$ mm at $z = 50$ mm	0.50	0.30	0.44	0.30

How to read it. For non-axisymmetric perturbations the kernels are good: correlation 0.84–0.99, slope 0.74–1.03. The worst case, around both walls, is the 60 µm short-wavelength field, with a

slope of 0.74: there the response is no longer linear.

For changes of the azimuthal mean profile the kernels are poor. For the gentle bump the correlation is only 0.44–0.50 and the slope 0.30, although the exact response to this bump is linear in its height between 5 and 30 µm (checked separately), so the fault is in the kernels and not in the linearity of the guide. For the sharp step the kernels get the mean change of the transmission right to 1 % but not its radial pattern: correlation 0.13–0.27.

The reason is a cancellation. A smooth deformation is a sum over many z knots, and the responses of the single knots are large and nearly cancel: the response to the bump is about 3 % of the sum of their magnitudes. Errors of a few percent in the single-knot kernels (third-order terms of the finite difference, Monte Carlo noise) therefore become errors of order one for smooth shapes.

A study with a smaller finite-difference amplitude, 15 µm instead of 60 µm (`kernel_modes.py ... 0.015`, in the chain `LG_KAMP=0.015`; log in `results/checks/test_jacobian_amp015.log`), gives 0.86–1.00 for the non-axisymmetric fields, 0.66–0.80 for the gentle bump (slope 0.62–0.66) and 0.26–0.48 for the step. That is better but not good. The published result was *not* rerun with it; the default stays 60 µm.

What follows from this.

- The fit finds steps along the mean profile inefficiently. The quoted fit quality is not affected, because every trial step is accepted or rejected after a full ray trace (Section 2.8). It is one more reason why the mean profile is poorly fixed (Section 9), and why the fit may stop short of the best wall the model allows.
- The resolution study (Section 6.7) uses the same kernels. Its figures are only indicative for broad smooth shapes.
- The two statements of non-uniqueness do not depend on the kernels. Both are statements of existence, checked with full ray traces: in the closure test a wall different from the true one fits the synthetic map to the noise, and two different walls fit the measured scan equally well (Section 6.8).

6.6 Closure test

```
../tools/run_limited.sh ./closure.sh 1
python make_closure_fig.py ../out/synth_cl1.npz ../out/final_cl1.npz ../results/en en
```

`make_synth.py` builds a known wall: the Stage-A profile plus seven Gaussian bumps and dents (0.15–0.3 mm high, 2–4 mm wide along the arc, 6–12 mm long). The truth lives on a 48×23 spline grid, twice as fine in azimuth and half as fine along z as the 24×43 grid of the fit. It is smooth, and the fit grid represents it to about 0.1 µm, so the test probes what the scan determines, not what the grid can represent. The script traces a synthetic scan and adds the measured point noise. `closure.sh` then runs `pipeline.sh` on it and finishes with the resolution study.

The reconstruction is given the entrance aperture of the synthetic wall itself, as it is given the measured aperture for the real scan: `make_synth.py` stores the Fourier coefficients ($m = 0 \ldots 24$) of that aperture in the synthetic-scan file, and `load_scan.aperture_fourier` returns them whenever `LG_SYNTH` is set. Earlier versions imposed the measured aperture on the synthetic wall, whose entrance differed from it by 38 µm RMS against a constraint of 5 µm; that penalised the true wall and made the test unfair.

The outcome (Figure 4) has two parts and both matter. The synthetic map is reproduced to the noise: residual 0.78 %, $\chi^2/N = 0.95$. The chain can therefore reach the noise level when the truth lies inside the model. The wall is a different matter: true and recovered local unevenness are only weakly correlated, at 0.18 (1 would be a perfect recovery), the recovered amplitude is 7 µm against

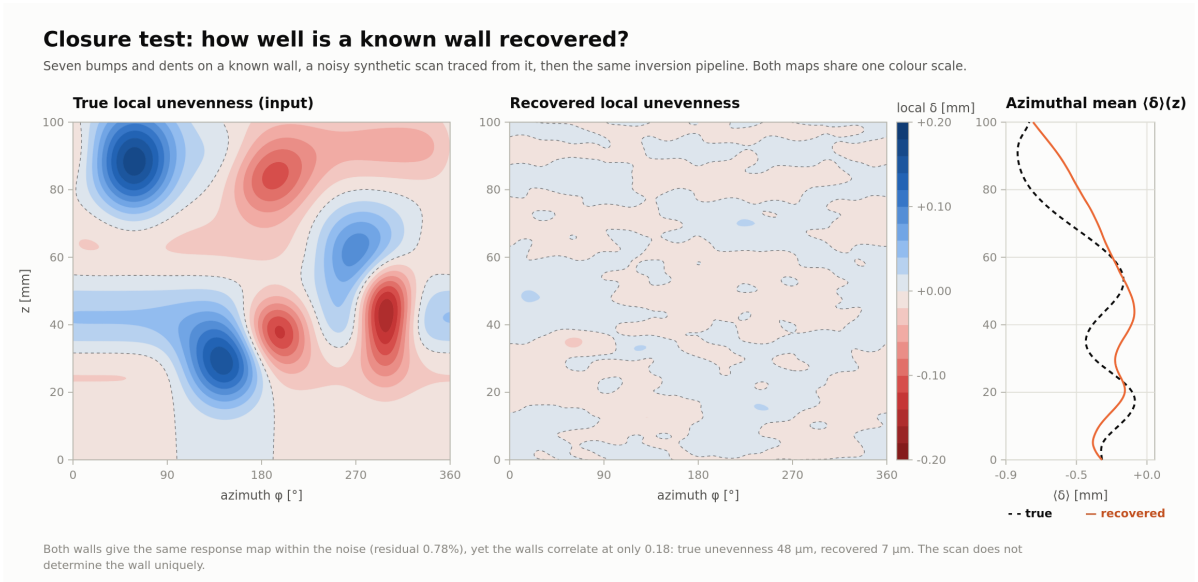


Figure 4: Closure test. Left: the true local unevenness put in. Middle: the unevenness recovered from the synthetic scan, on the same colour scale. Right: azimuthal mean profiles. Both walls give the same response map within the noise.

48 µm true, their difference is 47 µm, and the azimuthal mean profile is off by 172 µm RMS with a mean offset of +104 µm. All these are root mean squares: a bias counts.

6.7 Resolution study

```
python study_resolution.py cl1b ../out/synth_cl1.npz ../out/fitB_cl1b.npz
```

The Jacobian, linearised around the axisymmetric base wall, is decomposed by singular values after the direction of the map itself has been projected out (the overall amplitude is a free parameter of every fit). The decomposition orders the wall patterns by how strongly they change the scan. The scan fixes the amplitude of pattern k to $\pm\sigma_T/s_k$ (one standard deviation), with s_k its singular value and σ_T the noise of one scan point in transmission units. Table 3 gives, for the k best-determined patterns, that uncertainty for pattern k expressed as the RMS height of the pattern on the wall surface, the share of the true test wall's deformation power that lies in them, the correlation between the true and the recovered wall inside them, and the ratio of the recovered to the true amplitude there.

Table 3: What the scan determines (closure test). For the k best-determined patterns: one-sigma uncertainty of the amplitude of pattern k as surface RMS, share of the true wall's deformation power, correlation of recovered and true wall, ratio of recovered to true amplitude. The singular values fall smoothly, by three orders of magnitude over the 1032 patterns.

Best k	Uncertainty [µm]	Share [%]	Correlation	Amplitude ratio
10	0.021	0.2	+0.94	1.07
30	0.033	1.4	+0.97	0.92
60	0.031	1.6	+0.95	0.90
100	0.027	1.7	+0.91	0.90
150	0.039	4.0	+0.70	0.67
200	0.22	7.6	+0.49	0.54
300	0.88	13.1	+0.28	0.49
500	1.03	28.3	+0.18	0.37

The singular values fall smoothly: there is no sharp boundary between patterns the scan sees and patterns it does not, and no single number of visible patterns. With explicit cut-offs: inside the best 30 patterns the recovered and the true wall correlate at 0.97, and these patterns hold 1.4 % of the test wall's deformation power; inside the best 100 the correlation is 0.91 (1.7 %), inside the best 150 it is 0.70 (4.0 %). Correlation alone hides amplitude errors: inside the best 100 patterns the recovered amplitude is 0.90 times the true one. The best-determined patterns are fixed to well below a micrometre of surface RMS (0.033 µm for pattern 30, 0.027 µm for pattern 100), but they hold little of the deformation of a wall with broad, gentle bumps. The supported statement is therefore: the wall is recovered well only inside roughly the 100 best-determined combinations, which hold about 1.7 % of the true deformation.

Four caveats. The Jacobian used here is the one of the kernels, which is poor for broad smooth shapes and for the mean profile (Section 6.5); the figures of Table 3 are therefore indicative, not precise. The analysis is linear, around the axisymmetric base wall, not around the fitted wall. The patterns are orthonormal in the metric of the spline-coefficient vector, not in a metric of the surface. And where a pattern drops out of sight depends on its amplitude, so any cut-off k is a choice.

6.8 Two fits of the measured scan

```
python compare_walls.py ../out/final_data.npz ../results/earlier/wall_coeffs_cb81f08.
    json
```

The closure test shows the non-uniqueness on a synthetic scan. This check shows it on the measured one. `results/earlier/wall_coeffs_cb81f08.json` holds the wall that was published with an earlier version of the toolkit (commit `cb81f08`), before the march and the ray sequence of the tracer were corrected. The script traces that wall with the present tracer, aperture and light spot (20 000 rays per point by default; third argument), fits its amplitude as `final.py` does, and compares it with the wall of the present result. It writes `../out/compare_prev.json`, which `collect_numbers.py` copies into the block `prev` of `numbers.json`; `run_all.sh` runs it right after the pipeline. Both walls must be on the same spline grid.

Field	Value	Meaning
`rms_earlier`, `rms_now`	0.92 %, 0.90 %	data minus simulation in the inner aperture, earlier and present wall
`map_diff`	0.28 %	RMS difference of the two simulated maps
`corr_local`	0.15	correlation of the local unevenness (azimuthal mean removed), whole wall
`bands`	0.80; +0.02, −0.49, −0.19, −0.26	the same in five 20 mm bands of z, from the entrance to the exit
`loc_rms_earlier`, `loc_rms_now`, `loc_diff`	13.5, 12.9, 17 µm	RMS local unevenness of the two walls and of their difference
`prof_diff`, `prof_diff_max`	225, 516 µm	RMS and largest difference of the azimuthal mean profiles

How to read it. The two walls fit the measured scan equally well, and their simulated maps differ by much less than either differs from the data. If the scan determined the wall, the two would agree. They agree only in the first 20 mm behind the entrance (correlation 0.80); further in, the local unevenness is uncorrelated or even opposite, the difference of the two (17 µm RMS) is larger than either unevenness, and the mean profiles differ by up to 516 µm. A correlation near 1 here would be a necessary sign of a determined wall, not a sufficient one; the low value found is direct

evidence, on the measured data, that the wall is not determined. The two fits are not independent samples of all walls that fit (same method, same regularisation, same start recipe), so the spread between them is a lower bound on the freedom the scan leaves.

6.9 Further studies

Script	Purpose
`study_deformations.py`	response maps for a set of simple test deformations (barrel, hourglass, single bumps, entrance flare, exit pinch)
`study_reflectivity.py`	radial profile of the ideal cone for mirror reflectivities below 1, against the data
`study_launch_cone.py`	checks that the launch-angle cut `smax` loses no detected light
`look_data.py`	quick plots of the raw scan

7 Outputs and file formats

7.1 Input scan

`data/scan_summary.csv` has one line per scan point with the columns `idx_x`, `idx_y`, `direction`, `cmd_x_mm`, `cmd_y_mm`, `pos_x_mm`, `pos_y_mm`, `timestamp`, `n_events`, `n_saved`, `probe_V`, `passed`, `acq_time_s`. The grid indices `idx_x`, `idx_y` place each point; the code uses the positions `pos_x_mm`, `pos_y_mm`, the signal `probe_V` (volts) and the flag `passed`. The grid must be regular and complete.

7.2 Intermediate files in `out/`

File	Arrays
`edge.npz`	`cx`, `cy` centre (stage coordinates); `ped`, `ped_rms` pedestal; `fourier[9,2]` cosine and sine coefficients ($m = 0 \dots 8$) of $r_{\mathrm{ap}}(\varphi)$; `phi`, `r_edge`, `sigma`, `level` per-sector edge fits; `spot_radius`, `dr_disc` from `edge_spot.py`
`A_<tag>.npz`	`p` profile coefficients (and the reflectivity if fitted); `NZ`; `rho`, `dat`, `err`, `T`, `A` the fitted radial profile; `zz`, `d0` the profile $\delta_0(z)$
`start_<tag>.npz`	`c[24, NZ+3]`, `NZ`, `NPHI`, `refl`
`kmodes_<tag>.npz`	`A[nk, 13, 3, nrho]` harmonic amplitudes; `QS[13,3]` the harmonics q of each mode (-1 = unused); `rho`; `p` base profile; `T0` base map; `NPHI`, `NZ`, `refl`
`fitB_<tag>.npz`	`c[24, 43]` fitted coefficients; `base`; `A` amplitude; `hist` (χ^2/N, residual, amplitude per iteration); `T`, `xs`, `ys`, `dat` model, positions and data of the fitted points; `lam`, `NPHI`, `NZ`, `refl`
`final_<tag>.npz`	`x`, `y` scan axes; `S[nx,ny]` pedestal-subtracted data; `M` model; `T` transmission; `Mideal` ideal cone; `A`; `R` distance from the centre; `cx`, `cy`; `phi[361]`, `z[201]`, `DEL[361,201]` the wall on a grid; `c`; `refl`; `mc` model noise
`synth_<tag>.npz`	`S` synthetic scan; `c_true[48,23]`; `bumps`; `A`; `fourier[25,2]` aperture of the synthetic wall; `seed`; `base_sha1` fingerprint of the profile it is built from
`pipeline_<tag>.log`	full output of every step of `pipeline.sh`
`ideal_cone.json`, `gpu_check.json`, `recip.json`	results of `test_ideal_cone.py`, `test_gpu_vs_cpu.py`, `test_reciprocity.py`; the last two with the fingerprint `wall_sha1` of the wall
`g4_compare.json`, `g4_ideal.json`	Geant4 against the tracer for the fitted wall and the ideal cone (`ratio`, `ratio_err`, `pull_rms`, `corr`, `spot`, `refl`, `wall_sha1`)
`resolution.json`	resolution study: `k`, `corr`, `amp_ratio`, `frac_true`, `amin_um`, `amin_surf_um`, `corr_all`, `base_sha1`
`A_refl.npz`	Stage A with the mirror reflectivity free, started from `A_data.npz` (`run_all.sh --with-checks`)
`jacobian.json`, `test_jacobian.log`	`test_jacobian.py`: kernels against exact changes of the map (Section 6.5); the JSON file carries `wall_sha1`. The logs of the two runs quoted in this manual are tracked in `results/checks/`
`compare_prev.json`	`compare_walls.py`: this wall against the wall of an earlier fit (Section 6.8), with `wall_sha1` and `earlier_sha1`

7.3 Results

`wall_delta.csv`: three comment lines, then the columns `phi_deg` (0–359, step 1), `z_mm` (0–100, step 0.5), `r_cad_mm`, `delta_mm`, `delta_azimuthal_mean_mm`, `delta_local_mm`.

`wall_coeffs.json`: `NPHI`, `NZ` and the coefficient matrix `c[NPHI][NZ+3]` in mm. It can be fed back to the tracer:

```
import json, numpy as np, lgtrace as lg
c = np.array(json.load(open('../results/wall_coeffs.json'))['c'])
T = lg.transmission([0.0, 2.0], [0.0, -1.0], coeffs=c, nray=20000, smax=0.55, sig
    =-0.34)
```

`sig` < 0 is a disc of radius $-$`sig`; `sig` > 0 a Gaussian sigma; positions are relative to the aperture centre.

`results/earlier/wall_coeffs_cb81f08.json`: the wall published with an earlier version of the toolkit, in the format of `wall_coeffs.json`; the reference of `compare_walls.py`.

`numbers.json`: the fingerprint of the wall (`wall_sha1`), the fit quality (`rms`, `bias`, `rms_id`, `chi2`, `corr`, `rms_ring`, `bias_ring`, `ring_worst`, `excess`, in percent of the plateau; `ring_worst` is the most negative azimuthal mean of data minus simulation in the 0.1 mm annuli between 8.0 and 8.5 mm), the noise (`noise` of one scan point, `mc_noise` of the model map, in percent), the wall statistics (`loc_rms`, `loc_min`, `loc_max` in µm; `mean_min`, `mean_max` in mm), the aperture (`r_ap`, `spot_r`), and one block per check that has been run: `ideal`, `g4_ideal`, `g4`, `gpu`, `recip`, `jacobian` (`corr_min`, `corr_max`, `slope_min`, `slope_max` for the non-axisymmetric perturbations; `m0_step_corr_min/max`, `m0_smooth_corr_min/max`, `m0_smooth_mean_min/max` for the axisymmetric ones), `refl_fit`, `prev`, `closure`, `resolution`. Every "rms" is a root mean square, with the bias given separately where it matters. A block carries `current`: true if the check was run on the wall of this result, false if its file is left over from an earlier reconstruction.

Geant4 mesh (`mesh.txt`): first line "`NPHI NZ`"; then `NZ` lines, each with z and the `NPHI` wall radii at $\varphi_i = 2\pi i/\texttt{NPHI}$, in mm.

8 Adapting the toolkit to another light guide or scan

Geometry. The cone is defined by three constants, which appear in four places and must agree:

- `src/lgtrace.py`: `R1, R2, L` (and `SLOPE` derived from them);
- `src/lgcl.py`: the macros `R1`, `LEN`, `SLOPE` in the OpenCL source (`SLOPE` is written out as a number);
- `web/lgtrace.js`: `R1, R2, L` for the browser demonstration;
- `g4/lgsim.cc`: `R1, R2, LEN`, and the sizes of the world box and of the two photon dumps if the guide is much larger.

`export_wall.py` and `make_ray_svg.py` also write out $r_{\mathrm{CAD}} = 8.75 - 0.0475\,z$. A shape that is not a straight cone needs a different reference surface in the functions `_f` and `_fg` and in the bounding-cone bracket; the B-spline part carries over unchanged.

Scan. Any regular rectangular grid works. Either provide a CSV with the same columns and point `LG_SCAN` to it, or edit `load()` in `src/load_scan.py`; it must return the axes `x`, `y` and the signal `V[ix, iy]`. In `src/edge.py` set the first guess of the centre and the radial windows used for the pedestal ($r > 10$) and for the edge fit ($7.6 < r < 9.6$).

Fit radius. `LG_RCUT` (8.0 mm) must stay inside the aperture by more than the spot radius, so that the spot is never clipped at a fitted point. The polar grid in `kernel_modes.py` (57 radii of 0.15 mm) must cover it.

Launch cone. `smax` must exceed the largest $\sin\theta$ of any detected ray, roughly R_2/R_1 plus a margin. Check with `study_launch_cone.py`. Too small a value silently loses light; too large a value only costs time.

Resolution of the wall. N_φ is an argument of `kernel_modes.py` and `refine.py` writes 24 azimuthal knots for a Stage-A input; N_z is the second argument of `refine.py`. Finer grids resolve thinner arcs but add poorly constrained parameters; cost grows with $N_z \times (N_\varphi/2 + 1)$ kernel runs.

Regularisation. λ is the third argument of `stageB.py`. With the damped iterations the result depends weakly on it: the fit stays close to the start wall anyway. To see the effect of the prior, change the start wall rather than λ.

Noise and statistics. The point noise of the scan (`SIGD` in `stageB.py`, also used in `final.py` and `collect_numbers.py`) is 2.0×10^{-4} V for the shipped data; estimate it for new data from

the high-pass residual of the plateau. Rays per point: 12 000 for the kernels, 20 000 per iteration, 40 000 for the final map. Halving them roughly halves the run time and raises the model noise by 30–40 %.

9 Limits and interpretation

The solution is not unique. Different walls give the same response map within the noise. Two fits of the measured scan reproduce it equally well (0.92 % and 0.90 %) with walls that agree only close to the entrance: correlation 0.15 overall, 0.80 in the first 20 mm (Section 6.8). In the closure test the true wall is recovered well only inside roughly the 100 shape combinations the scan determines best (correlation 0.91), which hold 1.7 % of the true deformation (Table 3); a wall with broad, gentle bumps 0.15–0.3 mm high (48 µm RMS) and the recovered wall with 7 µm RMS fit the same synthetic scan equally well. The wall shown is one regularised solution: the fit penalises unevenness and its damped iterations stay near the axisymmetric start wall, so it tends towards a smooth wall. It is not shown to be the least uneven wall that fits, since the stopping rule proves no minimum and Q is not the surface RMS. The local unevenness of 13 µm RMS shows that little unevenness is enough to explain the scan. It is neither an estimate of what the real wall has nor a bound on it from below: part of the structure could come from reflectivity or detector non-uniformity instead of geometry.

The azimuthal mean profile is poorly fixed. It is strongly degenerate with the overall signal scale, with the exit radius and with the mirror reflectivity, and different fits return different profiles. The present profile and that of the earlier wall are 225 µm RMS apart, up to 516 µm, while both fit equally well; in the closure test the recovered profile is off by 172 µm RMS; with the reflectivity free the axisymmetric fit moves to $R = 0.96$ and another profile. The linear kernels are poor for changes of the mean profile (Section 6.5), so the fit finds such steps inefficiently. Common to all fits are only the entrance radius (8.47 mm, measured directly from the scan edge) and a bore that is narrower than CAD along the whole length. The local unevenness is 12.9 µm RMS here and 13.5 µm in the earlier wall; a still earlier fit from another starting point gave 44 µm. All reproduce the scan to about 1 %.

Everything is attributed to geometry. By assumption the mirror reflectivity is 1, the PMT response is uniform in position and angle, and the source is an ideal Lambertian disc. Real reflectivity variations, the seam of the reflective film, dust, or photocathode non-uniformity would all be read as wall unevenness.

Unexplained structure. The fit to the shipped scan ends at $\chi^2/N = 1.27$, not 1: a few tenths of a percent of correlated structure on a 1 mm scale is not explained by this fit. The closure test shows that the chain can reach the noise when the truth lies inside the model. It does not show that the fit of the shipped scan is the best one the model allows: two earlier fits of the same scan, with earlier versions of the code, ended at $\chi^2/N = 1.48$ and 1.31, so the value depends on the start, the settings and the code version. The remainder may be structure outside the model (smooth wall, perfect mirror, uniform detector), an unfinished optimisation, or both. χ^2/N is not a calibrated goodness of fit, because the effective number of fitted parameters is unknown.

The entrance rim. Between $r = 8.0$ mm and the aperture edge (8.47 mm) the measurement lies below the model. Over the ring from 8.0 to 8.5 mm the residual is about 3 % RMS per point with a mean of -1.8 % of the plateau, and the azimuthal mean reaches -3.6 % in the worst 0.1 mm annulus. The measured signal drops earlier than the model inside the aperture but has no tail outside, as if a band about 0.1 mm wide at the rim were lossy (a chamfer, or film that stops short of the edge). This is below the resolution of the model and the ring is excluded from the fit.

Resolution. The deviation field has 2.5 mm cells along the axis and 15° in azimuth (2.2 mm of arc at the entrance, 1.0 mm at the exit). Finer structure is smoothed.

Orientation of the azimuth. φ follows the scan axes. Whether the figure is a mirror image of the hardware depends on the stage orientation.

What would pin down the real wall. Independent information, each piece constraining a further set of shape combinations:

- scans with a collimated beam at several incidence angles (each angle selects different ray families, where the diffuse source averages over all of them);
- angle-resolved or position-resolved readout at the exit instead of one integrating PMT;
- a scan from the exit side;
- direct profilometry of the wall, or a coordinate measurement of the bore at a few heights, to fix the azimuthal mean.

The resolution study (`study_resolution.py`) can be repeated for a simulated additional measurement to see how many more patterns it would make visible before taking the data.

10 Troubleshooting

`IndexError` or "no devices" when the GPU tracer starts

`pyopencl` found no GPU device. Check `clinfo -l`; make sure the user is in the `render` and `video` groups; on AMD check that the ROCm OpenCL ICD is installed. To run without a GPU set `LG_GPU=0`.

The desktop stutters or the GPU driver resets during a run

The tracer submits slices of about 0.6 s (`slice_s` in `lgcl.transmission`). Lower it if the GPU is shared with interactive work. Run only one GPU job at a time.

Results change after editing `lgtrace.py`, or `numba` complains about a cached function

Delete `src/__pycache__`. The compiled functions are cached there and rebuilt on the next run (a few seconds).

A driver stops with "step failed" or a FAIL line

The shell drivers stop at the first failing step or self-test. `pipeline.sh` prints the end of `out/pipeline_<tag>.log`; the full output of the step is in that file.

A job dies with exit code 137, or "Killed"

It exceeded the memory limit of the wrapper. The pipeline needs less than 1 GB; a Geant4 job with the 720 000-facet mesh about 0.7 GB. Raise the limit with `LG_MEM=12G` only if the job is known to need it.

"no improving step; stop" in the first iteration

See Section 5: raise `LG_MU`, or recompute the kernels around the current wall. At the end of a run the message is the normal stopping rule.

`stageB.py` stops, then the final χ^2 is higher

`final.py` uses a different seed and more rays; differences of a few hundredths in χ^2/N are Monte Carlo noise. Larger differences mean the fit followed the noise of its own ray set; raise `LG_NRAY`.

The manual figures come out empty, or Firefox refuses to start

Another Firefox is using the same profile. `make_manual_figs.sh` keeps a private profile in `docs/manual/.ffprofile`; delete that directory and run again.

Geant4: "Overlapping daughter with mother volume"

The world box in `lgsim.cc` must contain the cavity and both photon dumps; enlarge it when the guide is longer than 100 mm.

Geant4: "Particle e- without a Process Manager"

This appears when `G4OpticalPhysics` is registered alone. Keep the minimal physics list of `lgsim.cc`: the optical photon only, with transportation and the optical boundary process (`G4OpBoundaryProcess`).

Geant4 is very slow

Tracking in the tessellated solid costs about 10 ms per photon whatever the mesh size between 180 000 and 720 000 facets. Use few points with a few thousand photons, in parallel processes.

11 Resource policy: `tools/run_limited.sh`

The development machine is shared, so every long or parallel job is started through a wrapper that keeps it from exhausting the memory and makes it yield the CPU to other work.

```
tools/run_limited.sh <command> [arguments...]
tools/run_limited.sh --cores        # print the number of cores a job would get now
```

- One invocation is one transient `systemd` user scope with `MemoryMax` (default 6 GB) and no swap: a job that exceeds the limit is killed.
- The number of cores is chosen when the job starts: all cores, minus those busy with other work, minus four spare, clamped to the range 2–24. It sets the CPU quota of the scope and the thread counts of `numba`, OpenMP and BLAS.
- The job runs at `nice 19` with idle I/O priority and a CPU weight of 10, so other work gets the CPU first.
- `LG_GPU=1` is exported, so tracing runs on the GPU unless `LG_GPU=0` is given.

Overrides: `LG_MEM=12G`, `LG_CPUS=8`, `LG_GPU=0` in front of the command. On a machine without `systemd` user sessions run the commands directly and set `LG_GPU=1` and `NUMBA_NUM_THREADS` by hand.

What the wrapper does not do. The limits apply to one invocation: jobs started side by side each get their own scope and their own limits, so parallel work that needs a total cap must be started from inside *one* invocation. `g4/run_final.sh` does that for its Geant4 processes. And a cgroup memory limit does not cap GPU allocations: the GPU is outside the cgroup accounting. The tracer therefore limits itself, with host buffers of a few megabytes and sub-second slices. Run one GPU job at a time.

12 Author, contact, license and citation

LGWallRecon was written by Hengne Li (李衡讷), South China Normal University.

Contact: `Hengne.Li@cern.ch`, `Hengne.Li@m.scnu.edu.cn`.

Repositories:

- GitHub: `https://github.com/hengne/LGWallRecon-Claude-Fable52`
- GitLab (SCNU group): `https://gitlab.cern.ch/SCNU/LGWallRecon-Claude-Fable52`

License: LGWallRecon is released under the MIT License, Copyright (c) 2026 Hengne Li (李衡讷). You may use, copy, modify and redistribute it, including the measured scan and the CAD file in `data/`, provided the copyright and permission notice is kept. It comes without warranty. The full text is in the file `LICENSE` at the top of the repository.

To cite the toolkit:

> Hengne Li, *LGWallRecon: light-guide inner-wall reconstruction from an entrance-face scan*, software, 2026.
> `https://github.com/hengne/LGWallRecon-Claude-Fable52`

When quoting results obtained with it, state the assumptions of the model (perfect specular wall, uniform detector, Lambertian source) and that the reconstructed wall is one regularised solution of a non-unique inverse problem.